

UNIVERSIDAD TECNOLÓGICA METROPOLITANA (UTM)

ESTUDIA CACO - GUÍA Y DOCUMENTACIÓN COMPLETA DE EXAMEN

Asignatura: Programación Orientada a Objetos | Docente: Mtra. Marlene Ruiz Barboza | Alumno: Mario Isaac Ornelas

DOCUMENTO OFICIAL DE ESTUDIO - ESTUDIA CACO:

Este documento técnico contiene el desglose completo de la arquitectura del Sistema de Gestión de Salón de Belleza 'Nicté Ha'. Incluye la explicación detallada, líneas de código, snippets C#, Constructores, Instancia única, Gestión de Memoria en el HEAP / Garbage Collector, Principios SOLID, Módulo de Seguridad, 2 Herencias, 2 Encapsulamientos, Abstracciones, Polimorfismos y Patrones de Diseño (Singleton, Factory Method y Observer con sus 3 observadores concretos).

Repositorio Oficial GitHub: https://github.com/marioornelas345-del/Ordinario_POO.git

Concepto / Requerimiento Examen UTM	Ubicación y Rangos de Líneas	Código / Snippet Relevante	CÓMO se aplicó	POR QUÉ se usó
Constructores (Públicos y Privados)	Models/Cita.cs (Líneas 25-28) Patrones/Singleton/BackupManager.cs (Línea 17)	<pre>public Cita() { Estado = 'Agendada'; } private BackupManager() { }</pre>	Se implementan constructores públicos para inicializar el estado por defecto de las entidades y constructores privados para restringir la creación con 'new'.	Garantiza la inicialización segura de los objetos y previene que clases Singleton sean instanciadas erróneamente desde fuera.
Instancia e Instanciación (Punto de Acceso Global)	Patrones/Singleton/BackupManager.cs (Líneas 20-33) Controllers/AdminController.cs	<pre>public static BackupManager Instancia { get { lock(_lock) { return _instancia ??= new BackupManager(); } } }</pre>	Se expone una propiedad estática pública 'Instancia' que devuelve el único objeto instanciado en el sistema con cerrojo de hilos (Thread-safe).	Permite que toda la aplicación comparta una sola referencia activa del administrador de respaldos y configuración del salón.
Gestión de Memoria en HEAP (Destructores y Garbage Collector)	Models/Cita.cs Models/Cliente.cs (Gestión Runtime CLR)	<pre>// Asignación Dinámica en HEAP: var cita = new Cita(); // Recolección Automática en HEAP: GC.Collect(); // Administrado por el CLR</pre>	Todos los objetos instanciados (Cita, Cliente, Estilista) se alojan dinámicamente en la memoria HEAP. C# administra su liberación mediante el Garbage Collector.	En .NET Core moderno se evita el uso de destructores manuales que bloquean hilos, confiando la memoria HEAP al reclamo automático y eficiente del Garbage Collector.
Principios SOLID (SRP, OCP, LSP, ISP, DIP)	Controllers/AdminController.cs (Líneas 22-26)	<pre>// SRP: Control exclusivo de flujos admin. // OCP: Abierto a extensión via Interfaces. // DIP: Inyección de DbContext y contratos.</pre>	Se aplican los 5 principios SOLID en la arquitectura de controladores, servicios e inyección de dependencias de la base de datos.	Asegura un código escalable, mantenible, fácil de probar y libre de acoplamiento rígido cumpliendo los estándares de POO de la UTM.
Módulo de Seguridad y Bloqueo (Filtro de Sesión y Gatekeeper)	Controllers/HomeController.cs (Líneas 15-50) Program.cs (Middleware)	<pre>if (HttpContext.Session.GetString('UsuarioLogueado') == null) { return RedirectToAction('Index', 'Home'); }</pre>	Se implementó un middleware de sesión que bloquea el paso a cualquier vista del sistema si el usuario no se ha autenticado previamente.	Protege la aplicación garantizando que solo usuarios registrados o administradores puedan navegar por el sistema.
Selector Cuentas de Google (Google OAuth Selector)	Views/Auth/Login.cshtml (Líneas 35-140)	Continuar con Google	Un modal desplegable permite elegir cuentas de Google configuradas (Admin o Cliente) e iniciar sesión en 1 clic.	Proporciona una interfaz limpia y rápida para la autenticación sin necesidad de credenciales complejas.
Separación de Vistas por Roles (Admin vs Cliente RBAC)	Views/Shared/_Layout.cshtml (Líneas 25-60) Controllers/AdminController.cs	<pre>if (esAdmin) { /* Muestra solo Panel Administrador */ } else { /* Muestra solo Servicios, Citas, Pagos, Ubicación, Perfil */ }</pre>	El menú de navegación y las rutas del sistema se dividen estrictamente según el rol autenticado (Administrador vs Cliente).	Evita que un cliente acceda al panel administrativo o que el administrador vea pestañas no relevantes para su gestión.
Encapsulamiento Tipo 1 (Private Set)	Models/Cita.cs (Líneas 12 y 32-37)	<pre>public DateTime FechaHoraFin { get; private set; } public void AsignarHorario(DateTime hora) { ... }</pre>	Protege la propiedad FechaHoraFin con 'private set' permitiendo su modificación únicamente desde el método AsignarHorario().	Garantiza la regla de negocio del examen: ninguna cita puede durar más o menos de 1 hora exacta.
Encapsulamiento Tipo 2 (Private Constructor)	Patrones/Singleton/BackupManager.cs (Líneas 13-17)	<pre>private BackupManager() { }</pre>	Restringe la creación del objeto mediante un constructor privado.	Encapsula la creación del administrador de respaldos para forzar el uso del Singleton.

Abstracción 1 (Clase Abstracta Persona)	Models/Persona.cs (Líneas 6-11)	<pre>public abstract class Persona { public int Id; public string Nombre; ... }</pre>	Define la clase base abstracta Persona con las propiedades fundamentales compartidas.	Abstrae la entidad humana reduciendo código duplicado entre Clientes y Estilistas.
Abstracción 2 (Interfaces de Dominio)	Patrones/Factory/NotificacionFactory.cs (Líneas 10-14) Patrones/Observer/CitaObserver.cs	<pre>public interface INotificacion { void Enviar(string msg); } public interface IObservadorCita { void Actualizar(...); }</pre>	Establece contratos puros con métodos abstractos obligatorios para notificaciones y observadores.	Desacopla la lógica del sistema permitiendo agregar nuevos tipos de mensaje sin alterar el código existente.
Herencia 1 (Cliente)	Models/Cliente.cs (Líneas 7-15)	<pre>public class Cliente : Persona { public string Telefono { get; set; } }</pre>	Cliente hereda de la clase base Persona adquiriendo Id, Nombre y Correo.	Reutiliza la estructura base e incorpora el teléfono de contacto requerido para citas.
Herencia 2 (Estilista)	Models/Estilista.cs (Líneas 7-15)	<pre>public class Estilista : Persona { public string NumeroEmpleado { get; set; } }</pre>	Estilista hereda de la clase base Persona añadiendo datos laborales.	Demuestra el segundo tipo de herencia exigido para la gestión de empleados del salón.
Polimorfismo 1 (Factory)	Patrones/Factory/NotificacionFactory.cs (Líneas 16-33)	<pre>public class NotificacionSms : INotificacion { ... } public class NotificacionCorreo : INotificacion { ... }</pre>	Las clases concretas sobrescriben el método Enviar() de la interfaz INotificacion.	Permite enviar mensajes polimórficos sin importar si el destino es un celular o un e-mail.
Polimorfismo 2 (Observador SMS)	Patrones/Observer/CitaObserver.cs (Líneas 23-40)	<pre>public class ObservadorSmsCliente : IObservadorCita { public void Actualizar(...) { ... } }</pre>	Primer observador concreto que implementa el aviso vía SMS al cliente.	Cumple el primer canal de notificación automática exigido en el examen.
Polimorfismo 3 (Observador Correo)	Patrones/Observer/CitaObserver.cs (Líneas 42-60)	<pre>public class ObservadorCorreoCliente : IObservadorCita { public void Actualizar(...) { ... } }</pre>	Segundo observador concreto que implementa el aviso por e-mail al cliente.	Cumple el segundo canal de notificación de confirmación/cancelación de citas.
Polimorfismo 4 (Observador Bitácora)	Patrones/Observer/CitaObserver.cs (Líneas 62-80)	<pre>public class ObservadorBitacoraAdmin : IObservadorCita { public void Actualizar(...) { ... } }</pre>	Tercer observador concreto que audita los eventos en la bitácora del administrador.	Proporciona la trazabilidad completa requerida para supervisión del salón.
Modularidad Independiente	Estructura del Proyecto Carpetas: Controllers/, Models/, Views/, Patrones/, Data/	Carpetas separadas e independientes de Program.cs	Se organizan las clases en módulos independientes en la solución C#.	Cumple la indicación de modularidad separada de la clase principal.
Patrón Singleton (Backup BD)	Patrones/Singleton/BackupManager.cs (Líneas 1-55)	<pre>BackupManager.Instancia.GenerarRespaldoBaseDatos();</pre>	Punto de acceso global único para respaldos de base de datos SQL Server.	Genera copias físicas .bak en wwwroot/backups/ garantizando la integridad de datos.
Patrón Factory Method	Patrones/Factory/NotificacionFactory.cs (Líneas 1-55)	<pre>NotificacionFactory.CrearNotificacion('SMS');</pre>	Fábrica centralizada para crear objetos de notificación diaria.	Simplifica la instanciación de avisos para recordatorios del día siguiente.
Patrón Observer Completo	Patrones/Observer/CitaObserver.cs (Líneas 1-130)	<pre>SujetoCita.NotificarObservadores(citaId, estado, detalle);</pre>	El sujeto notifica automáticamente a los 3 observadores registrados.	Desacopla la lógica de agendado de los canales de salida de notificaciones.

Declaración de Conformidad Técnica:

Se certifica que el código fuente publicado en el repositorio oficial de GitHub https://github.com/marioornelas345-del/Ordinario_POO.git en la rama **main** cuenta con todos los Constructores, Instancias, Gestión de Memoria HEAP / Garbage Collector, Principios SOLID, Módulo de Seguridad y Patrones de Diseño exigidos en la rúbrica UTM.